

Course Code: CSE 440

Section: 01

Project Group No.: XX

Members:

1.Atyea Sanjeeda Ema (2031792642)

2.Saba Mahjamin (2121405042)

3.3rd Author (2222222)

4.4th Author (333333)

Strategic AI Agent Development for Texas Hold'em Using Expectiminimax and Monte Carlo Simulation

Atyea Sanjeeda Ema (2031792642)
Department of Electrical and Computer Engineering
North South University
Dhaka, Bangladesh

Saba Mahjabin (2121405642)
Department of Electrical and Computer Engineering
North South University
Dhaka, Bangladesh

Fahim Shahriar (2535154650)
Department of Electrical and Computer Engineering
North South University
Dhaka, Bangladesh

Sumaiya Akter (2131066642)
Department of Electrical and Computer Engineering
North South University
Dhaka, Bangladesh

Abstract—Poker is a difficult imperfect-information game that necessitates reasoning under ambiguity, strategic decision-making, and adapting to opponents' behaviours. This study describes the creation of a strategic AI agent for Texas Hold'em that uses the Expectiminimax search algorithm and Monte Carlo simulation to assess hand strength. The proposed system supports a variety of gameplay scenarios, including AI vs deterministic rule-based opponents, AI versus AI, and AI versus human players. In stochastic and partially observable contexts, the AI performs well by integrating decision-theoretic search and probabilistic evaluation. The modular architecture and hybrid approach give a lightweight, interpretable framework that is appropriate for game-theoretic AI research and academia. Performance measures like as win rates, decision distribution, and calculation time are assessed to ensure the system's effectiveness.

Index Terms—Poker, AI, Game Theory, Decision Making, Reinforcement Learning

I. INTRODUCTION

Poker is a typical example of a stochastic, imperfect-information game that presents substantial hurdles to AI research. Unlike deterministic games like chess or Go, poker necessitates reasoning under ambiguity, bluffing, and strategic adaptation to opponents' actions. The creation of AI bots capable of playing poker optimally has thus become a benchmark problem in computational game theory and decision-making research.

In recent years, breakthroughs in search algorithms and probabilistic modelling methods have allowed AI programs to outperform humans in a various types of poker variants. Counterfactual regret minimisation (CFR) [1], Monte Carlo Tree Search (MCTS) [2], and deep learning-based techniques [3], [4] have been successfully applied in both heads-up and multiplayer games. These methods allow the agents to test and examine large state spaces, simulate potential outcomes, and make effective intelligent decisions in real time very efficiently.

Our Paper describes the details of structural design and implementation of a Texas Hold'em poker AI that uses the

Expectiminimax algorithm and Monte Carlo simulation to access strength of hand and choose the moves that are best. This system is benchmarked in a various types of scenarios, including AI vs deterministic rule-based opponents simulation, AI vs AI simulation, and AI versus human players simulation. The initial contributions of this paper's works are:

- 1) A poker playing setting capable of handling a various types of opponents.
- 2) The integration of Expectiminimax search with Monte Carlo hand evaluation to manage uncertainty in not perfect information games.
- 3) Performance evaluation demonstrating the efficacy of the proposed AI methods in variety of gaming scenarios.

The prompt of the paper is figured as follows: One Section provides a review of related papers and their work and methodologies. One Section describes the methodology, game state representation, and algorithms used in the work. One Section represents the experimental setup and results (tables). One Section added conclusion to the paper and discusses future improvements.

II. LITERATURE REVIEW

Artificial intelligence (AI) research in imperfect-information games like poker has advanced dramatically over the last two decades, thanks to advancements in decision-theoretic reasoning, equilibrium-based optimisation, and simulation-driven evaluation. Poker is particularly difficult due to concealed information, stochastic card dealing, bluffing, and an exponentially huge game tree. As a result, ordinary minimax search is insufficient, and specialised algorithms have been created to handle uncertainty and strategic thinking.

A. Expectiminimax in Decision-Theoretic Search

By adding chance nodes, expectiminimax expands minimax and permits optimal reasoning in stochastic settings. The branching factor makes its direct application to poker computationally expensive, despite its widespread use in games like backgammon. However, expectiminimax serves as the

theoretical foundation for search-driven poker AI techniques and offers an organised probabilistic decision-making model. [5].

B. Equilibrium-Based Techniques: Poker Solvers and CFR

The introduction of Counterfactual Regret Minimisation (CFR), which approximates Nash Equilibrium in big imperfect-information games, was a significant breakthrough in poker AI [1]. CFR and its variations (CFR+, MCCFR) converge effectively, serving as the cornerstone for modern competitive poker agents.

CFR-based advancements led to the development of two significant poker AIs :

- **DeepStack** — uses deep neural networks and continuous subgame re-solving to achieve expert human-level performance [3].
- **Libratus** — Libratus used advanced abstraction, subgame solution, and post-processing refinement to overcome professional players in a 20-day competition [4].

These systems emphasised equilibrium computation in large-scale poker.

C. Monte Carlo and Probabilistic Methods

Monte Carlo methods (such as Monte Carlo Tree Search and probabilistic hand simulation) are commonly utilised in imperfect-information games. They evaluate decisions by sampling probable future deals and calculating predicted outcomes, making them suited for poker's unpredictable structure. [2].

Previously, Billings et al. shown that probabilistic reasoning mixed with simulation considerably enhances decision quality in poker agents, particularly in assessing hand strength and modelling hidden information. [6].

D. Frameworks for Poker Agent Development

General methodologies and architectures for designing competitive poker agents have been surveyed in [7] reviewed general approaches and architectures for developing competitive poker agents, emphasising hand-strength evaluation criteria, opponent modelling, abstraction and state representation, and simulation-based decision-making integration. These frameworks shape the creation of modern poker AIs and drove this project's concepts.

E. Positioning of This Work

Modern poker solutions typically use large-scale CFR or deep neural models, which can be costly to compute. This project proposes a lightweight, interpretable hybrid system that includes Expectiminimax search, Monte Carlo-based hand strength evaluation, and a modular Texas Hold'em environment for AI, deterministic bots, and human players. This contribution bridges the gap between traditional game-tree search methods and modern equilibrium solvers, resulting in a simple yet effective poker AI agent for academic settings with limited resources.

III. METHODOLOGY

This section presents the comprehensive framework for developing a Texas Hold'em poker simulator with Expectiminimax-based artificial intelligence. The methodology encompasses game environment design, strategic decision-making algorithms, heuristic evaluation functions, and performance testing protocols.

A. System Architecture

The poker simulation system consists of four primary components: (1) the game state manager, (2) the Expectiminimax search algorithm, (3) the Monte Carlo evaluation engine, and (4) the game orchestration layer. Fig. ?? illustrates the interaction between these components.

The architecture follows a modular design pattern where each component maintains clear interfaces, enabling independent testing and iterative refinement. The game state manager handles rule enforcement and state transitions, while the AI agent operates through the Expectiminimax algorithm to select optimal actions based on Monte Carlo simulations.

B. Game State Representation

The Texas Hold'em game state S is represented as a tuple:

$$S = \langle P, C_c, C_h, \beta, \pi, \phi, r \rangle \quad (1)$$

where $P = \{p_1, p_2, \dots, p_n\}$ represents the set of active players, C_c denotes the community cards, $C_h = \{c_1, c_2, \dots, c_n\}$ represents each player's hole cards, β is the current bet amount, π is the pot size, ϕ indicates the button position, and $r \in \{\text{preflop, flop, turn, river}\}$ represents the current betting round.

Each player p_i maintains a state vector:

$$p_i = \langle s_i, b_i, f_i, h_i \rangle \quad (2)$$

where s_i denotes the chip stack, b_i is the current bet, f_i is a binary fold indicator, and h_i represents the hole cards.

C. Expectiminimax Algorithm

The Expectiminimax algorithm extends the traditional minimax approach to handle the stochastic nature of poker by incorporating chance nodes for card dealing. The algorithm alternates between three node types: MAX nodes (AI decisions), MIN nodes (opponent decisions), and CHANCE nodes (card dealing).

1) *Value Computation*: The value function $V(s, d, t)$ for state s at depth d with node type t is defined recursively as:

$$V(s, d, t) = \begin{cases} E(s) & \text{if } d = 0 \text{ or } s \in S_{\text{terminal}} \\ \max_{a \in A(s)} V(s', d - 1, \text{MIN}) & \text{if } t = \text{MAX} \\ \min_{a \in A(s)} V(s', d - 1, t_{\text{next}}) & \text{if } t = \text{MIN} \\ \mathbb{E}_{c \sim D}[V(s_c, d - 1, \text{MAX})] & \text{if } t = \text{CHANCE} \end{cases} \quad (3)$$

where $A(s)$ represents the legal action set, s' is the resulting state after action a , $E(s)$ is the heuristic evaluation function,

S_{terminal} denotes terminal states, D represents the distribution of remaining cards, and t_{next} is determined by the next player's identity.

2) *Action Space*: The legal action space $A(s)$ for a given state includes:

$$A(s) = \{\text{fold, check, call, raise}(\alpha)\} \quad (4)$$

where raise amounts are constrained by:

$$\alpha = \beta + \max(\beta_{\text{BB}}, \pi) \quad (5)$$

with β_{BB} representing the big blind amount. This constraint limits branching factor while maintaining strategic depth.

D. Monte Carlo Hand Strength Evaluation

Hand strength estimation employs Monte Carlo simulation to compute win probability against random opponent holdings. For AI hole cards H_{AI} , community cards C_c , and remaining deck D_{rem} , the win probability P_{win} is approximated as:

$$P_{\text{win}}(H_{\text{AI}}, C_c) \approx \frac{1}{N} \sum_{i=1}^N \mathbb{I}[\text{rank}(H_{\text{AI}} \cup C_c^{(i)}) > \text{rank}(H_{\text{opp}}^{(i)} \cup C_c^{(i)})] \quad (6)$$

where N denotes the number of simulations (default: 1000), $C_c^{(i)}$ represents the completed board for simulation i , $H_{\text{opp}}^{(i)}$ denotes randomly sampled opponent cards, and $\mathbb{I}[\cdot]$ is the indicator function. Ties contribute 0.5 to the win count.

E. Heuristic Evaluation Function

The heuristic evaluation function $E(s)$ combines multiple strategic factors to estimate state value:

$$E(s) = \omega_1 \cdot V_{\text{pot}} + \omega_2 \cdot V_{\text{hand}} + \omega_3 \cdot V_{\text{pos}} + \omega_4 \cdot V_{\text{stack}} - \omega_5 \cdot C_{\text{call}} \quad (7)$$

1) *Expected Pot Value*: The expected pot value incorporates win probability and pot size:

$$V_{\text{pot}} = P_{\text{win}} \cdot \pi \quad (8)$$

2) *Hand Strength Value*: The hand strength component applies non-linear scaling to win probability:

$$V_{\text{hand}} = 100 \cdot P_{\text{win}} + B(P_{\text{win}}) - P(P_{\text{win}}) \quad (9)$$

where the bonus function $B(P_{\text{win}})$ rewards strong hands:

$$B(P_{\text{win}}) = \begin{cases} 40 & \text{if } P_{\text{win}} > 0.75 \\ 20 & \text{if } 0.65 < P_{\text{win}} \leq 0.75 \\ 0 & \text{otherwise} \end{cases} \quad (10)$$

and the penalty function $P(P_{\text{win}})$ discourages weak hands:

$$P(P_{\text{win}}) = \begin{cases} 60 & \text{if } P_{\text{win}} < 0.35 \\ 30 & \text{if } 0.35 \leq P_{\text{win}} < 0.45 \\ 0 & \text{otherwise} \end{cases} \quad (11)$$

3) *Position Value*: Position advantage is quantified based on distance from the button:

$$V_{\text{pos}} = 2 \cdot (|P| - d_{\text{button}}) \quad (12)$$

where $|P|$ is the number of active players and d_{button} represents the distance from button position.

4) *Stack Depth Factor*: Stack depth relative to pot size influences risk assessment:

$$V_{\text{stack}} = \begin{cases} 15 & \text{if } \frac{s_{\text{AI}}}{\pi} > 10 \\ -20 & \text{if } 2 < \frac{s_{\text{AI}}}{\pi} \leq 3 \\ -40 & \text{if } \frac{s_{\text{AI}}}{\pi} \leq 2 \\ 0 & \text{otherwise} \end{cases} \quad (13)$$

5) *Pot Odds Adjustment*: An additional penalty is applied for unfavorable pot odds:

$$C_{\text{call}} = \begin{cases} 1.8 \cdot c_{\text{call}} + 40 & \text{if } \frac{c_{\text{call}}}{\pi} > 0.5 \\ 1.8 \cdot c_{\text{call}} & \text{otherwise} \end{cases} \quad (14)$$

where c_{call} represents the chips required to call.

The weight coefficients are set as $\omega_1 = 1.5$, $\omega_2 = 1.0$, $\omega_3 = 1.0$, $\omega_4 = 1.0$, and $\omega_5 = 1.0$, balancing conservative play with strategic aggression.

F. Chance Node Sampling

For computational efficiency, CHANCE nodes employ sampling-based expectation estimation. Rather than exhaustively evaluating all possible card combinations, the algorithm samples M representative scenarios:

$$\mathbb{E}_{c \sim D}[V(s_c, d-1, \text{MAX})] \approx \frac{1}{M} \sum_{j=1}^M V(s_{c_j}, d-1, \text{MAX}) \quad (15)$$

where c_j represents sampled cards from the remaining deck distribution. The number of samples M varies by betting round: $M = 30$ for flop (3 cards), $M = 30$ for turn and river (1 card each), balancing accuracy and computational cost.

1) *Search Depth*: The Expectiminimax search employs a maximum depth of $d_{\text{max}} = 3$ plies, providing a balance between decision quality and computation time. At depth limit, the heuristic evaluation function estimates state value.

2) *State Cloning*: Each node expansion creates a deep copy of the game state to ensure search tree independence:

$$s' = \text{clone}(s) \oplus a \quad (16)$$

where $\oplus$ denotes the state transition operator applying action a to the cloned state.

3) *Terminal State Evaluation*: Terminal states receive exact evaluation based on game outcome:

$$E(s_{\text{terminal}}) = \begin{cases} \pi & \text{if AI wins} \\ -b_{\text{AI}} & \text{if AI loses} \\ E(s) & \text{if showdown required} \end{cases} \quad (17)$$

G. Algorithm Pseudocode

The complete Expectiminimax algorithm is presented in Algorithm III-G.

Expectiminimax Search for Poker

```

ExpectiminimaxSearchs  $A \leftarrow \text{getLegalActions}(s, \text{AI})$ 
 $|A| = 1$   $A[0]$   $\text{bestAction} \leftarrow \text{null}$   $\text{bestValue} \leftarrow -\infty$ 
 $a \in A$   $s' \leftarrow \text{clone}(s)$   $\text{applyAction}(s', \text{AI}, a)$ 
 $v \leftarrow \text{Expectiminimax}(s', d_{\max} - 1, \text{MIN})$   $v > \text{bestValue}$ 
 $\text{bestValue} \leftarrow v$   $\text{bestAction} \leftarrow a$   $\text{bestAction}$ 
Expectiminimaxs, d, t  $s \in S_{\text{terminal}}$   $\text{evaluateTerminal}(s)$ 
 $d = 0$   $E(s)$   $t = \text{MAX}$   $\text{MaxNode}(s, d)$   $t = \text{MIN}$ 
 $\text{MinNode}(s, d)$   $\text{ChanceNode}(s, d)$   $\text{MaxNodes}, d$ 
 $A \leftarrow \text{getLegalActions}(s, \text{AI})$   $\text{maxValue} \leftarrow -\infty$ 
 $a \in A$   $s' \leftarrow \text{clone}(s)$   $\text{applyAction}(s', \text{AI}, a)$ 
 $v \leftarrow \text{Expectiminimax}(s', d - 1, \text{MIN})$   $\text{maxValue} \leftarrow \max(\text{maxValue}, v)$ 
 $\text{maxValue}$   $\text{MinNodes}, d$ 
 $p \leftarrow \text{getCurrentPlayer}(s)$   $A \leftarrow \text{getLegalActions}(s, p)$ 
 $\text{minValue} \leftarrow \infty$   $a \in A$   $s' \leftarrow \text{clone}(s)$ 
 $\text{applyAction}(s', p, a)$   $\text{isBettingRoundComplete}(s')$ 
 $v \leftarrow \text{Expectiminimax}(s', d - 1, \text{CHANCE})$   $\text{nextPlayer}(s')$ 
 $p_{\text{next}} \leftarrow \text{getCurrentPlayer}(s')$   $t_{\text{next}} \leftarrow \text{MAX}$  if  $p_{\text{next}} = \text{AI}$  else  $\text{MIN}$ 
 $v \leftarrow \text{Expectiminimax}(s', d - 1, t_{\text{next}})$ 
 $\text{minValue} \leftarrow \min(\text{minValue}, v)$   $\text{minValue}$ 
 $\text{ChanceNodes}, d$   $D_{\text{rem}} \leftarrow \text{getRemainingCards}(s)$ 
 $n_{\text{cards}} \leftarrow \text{getCardsToDeal}(s.\text{round})$   $M \leftarrow 30$ 
Number of samples  $\text{totalValue} \leftarrow 0$   $j = 1$  to  $M$ 
 $s' \leftarrow \text{clone}(s)$   $C_{\text{sample}} \leftarrow \text{sample}(D_{\text{rem}}, n_{\text{cards}})$ 
 $s'.\text{communityCards} \leftarrow s'.\text{communityCards} \cup C_{\text{sample}}$ 
 $s'.\text{round} \leftarrow \text{nextRound}(s.\text{round})$   $\text{resetBetting}(s')$ 
 $v \leftarrow \text{Expectiminimax}(s', d - 1, \text{MAX})$   $\text{totalValue} \leftarrow \text{totalValue} + v$ 
 $\text{totalValue}/M$ 

```

H. Experimental Setup

The system evaluation comprises three testing scenarios:

- 1) **AI vs. Deterministic Players:** The Expectiminimax agent competes against rule-based opponents employing fixed strategies (tight-aggressive, loose-passive).
- 2) **AI vs. AI:** Multiple Expectiminimax agents with identical parameters compete to assess convergent behavior and strategy stability.
- 3) **AI vs. Human:** Human players interact with the AI agent through a command-line interface to evaluate perceived playing strength and decision quality.

Each scenario runs for 100 hands with starting stacks of 1000 chips, small blind of 10 chips, and big blind of 20 chips. Performance metrics include:

- Win rate (percentage of hands won)
- Average chip count trajectory
- Decision distribution (fold/check/call/raise frequency)
- Computation time per decision

I. Complexity Analysis

The computational complexity of the Expectiminimax algorithm is characterized by:

$$T(d) = O(b^{d-c} \cdot M^c) \quad (18)$$

where b represents the average branching factor (approximately 3-4 actions per decision node), d is the search depth, c is the number of chance nodes encountered, and M is the sampling rate per chance node. For $d = 3$, $b = 4$ and $M = 30$, the worst-case evaluation count is approximately $O(4^2 \cdot 30) = O(480)$ leaf evaluations per decision, which makes the approach computationally feasible for real-time play.

IV. RESULTS

To evaluate the effectiveness of the proposed poker agent, a series of controlled experiments were conducted against five predefined opponent profiles representing common playing behaviors in Texas Hold'em. Each type of opponent was tested under identical game conditions to ensure a consistent comparison. Tables I–III summarize the characteristics of the opponent strategies, the agent's performance against each other (AI vs AI), and the overall experimental outcomes. Both A1 and A2 are Expectiminimax-based AI agents with identical architectures but different search depth parameters. Across all strategic plays, Agent A2 consistently outperformed Agent A1, winning in every match. The results are presented in Table III

The reported metrics include win rate, profit, return on investment (ROI), number of hands played, and computation time, providing a comprehensive assessment of the agent's behavior under varying strategic conditions.

TABLE I
AI VS DETERMINISTIC PLAYERS SIMULATIONS

Opp.	Strat.	W(%)	Pft	ROI	H	LW	LL
Call St.	Always calls	66.7	990	99.0	6	950	40
Agg.	Constant raises	54.8	980	98.0	31	900	1080
Tight	Strong hands only	47.3	990	99.0	74	1080	900
Random	Random actions	51.3	980	98.0	39	1920	1080
Passive	Rarely raises	42.3	-1000	-100	26	360	1080

TABLE II
OVERALL EXPERIMENTAL SUMMARY

Metric	Value
Total Hands Played	176
Overall Win Rate	49.4%
Best Matchup	Calling Station
Worst Matchup	Passive Player
Most Profitable Opponent	Calling Station (+990)

V. CONCLUSION

This work presented a Texas Hold'em poker agent that combines expectiminimax search with Monte Carlo hand-strength estimation to operate effectively in stochastic and partially observable environments. The modular architecture enables probabilistic evaluation, heuristic scoring, and game-tree search to function as separate but complementary components within the decision pipeline. Experiments in several

TABLE III
AI VS AI SIMULATIONS (MONTE CARLO=5000)

Depth	Hands	Wins (A1,A2)	Chips (A1,A2)	Winner
(2,2)	5	(2,3)	(0,2000)	A2
(3,3)	3	(1,2)	(0,1920)	A2
(2,3)	5	(1,4)	(220,1780)	A2
(3,2)	5	(1,4)	(170,1830)	A2

game configurations, including deterministic rule-based opponents, AI self-play, and human interaction, indicate that the agent maintains consistent performance under uncertainty. The resulting framework is lightweight, interpretable, and well suited for research and academic study in game-theoretic artificial intelligence.

VI. FUTURE WORK

Future work may explore opponent-modeling mechanisms, improved Monte Carlo sampling strategies to reduce variance, and adaptive heuristics that respond to dynamic play conditions. Integrating reinforcement-learning methods is also a potential direction for enabling automated refinement of betting behavior over long-term interactions.

REFERENCES

- [1] M. Zinkevich, M. Johanson, M. Bowling, and C. Piccione, “Regret minimization in games with incomplete information,” in *Advances in Neural Information Processing Systems (NIPS)*, 2007, pp. 1729–1736.
- [2] D. Billings, N. Burch, A. Davidson, R. Holte, and J. Schaeffer, “Monte carlo methods in poker,” *Artificial Intelligence*, vol. 134, no. 1-2, pp. 91–145, 2002.
- [3] M. Moravčík, M. Schmid, N. Burch, V. Lisý, D. Morrill, N. Bard, T. Davis, K. Waugh, M. Johanson, and M. Bowling, “Deepstack: Expert-level artificial intelligence in heads-up no-limit poker,” in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017, pp. 406–415.
- [4] N. Brown and T. Sandholm, “Superhuman ai for heads-up no-limit poker: Libratus beats top professionals,” in *Proceedings of the 31st AAAI Conference on Artificial Intelligence (AAAI)*, 2017, pp. 645–651.
- [5] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2010.
- [6] D. Billings, D. Papp, and J. Schaeffer, “Using probabilistic knowledge and simulation to play poker,” in *AAAI Workshop on Computer Poker and Imperfect Information*, 1998.
- [7] D. Billings, N. Burch, A. Davidson, and J. Schaeffer, “Methodologies and tools for creating competitive poker agents,” in *Proceedings of the International Conference on Computers and Games*, 2002, pp. 13–28.